NAQLA

LAB 2

Lab 2 — The Booking API

Agentic Backend Engineering

Agentic Backend Engineering — Build Scalable Backends With and For AI Agents

Lab 2 — The Booking API

Overview

Goal

Yesterday you built Shahna's data model. Today you put an API in front of it. By 15:45 a customer can register, log in, list cities, get a price quote, book a shipment from that quote, list and view their own shipments, cancel one, and see its tracking timeline. Staff (dispatchers and admins) can view and cancel any shipment. Nobody else can.

Today's habit is **test-first with an agent**: the agent writes the failing test, you read it, *then* the agent writes the code. A test you have read is a specification you agree with. Code without a test is a claim you have not checked.

What you will build

Piece	Where it lives
`PricingService` with unit tests	`app/Services/PricingService.php`, `tests/Unit/PricingServiceTest.php`
Register and login (Sanctum tokens)	`app/Http/Controllers/Api/AuthController.php`
`GET /api/cities`	`app/Http/Controllers/Api/CityController.php`
`POST /api/quotes` with `StoreQuoteRequest`	`app/Http/Requests/StoreQuoteRequest.php`
Shipments: create, list, show, cancel, tracking	`app/Http/Controllers/Api/ShipmentController.php`, `app/Services/BookingService.php`
`ShipmentPolicy` (owner or staff)	`app/Policies/ShipmentPolicy.php`
`QuoteResource`, `ShipmentResource`, `ShipmentEventResource`	`app/Http/Resources/`
At least 20 passing tests	`tests/Unit/`, `tests/Feature/`

Starting point

```
git checkout day1-end
git switch -c my-day2
sail up -d
sail artisan migrate:fresh --seed
sail artisan test
```

All Day 1 tests must pass before you begin (the `day1-end` reference has **23**). If they do not, tell your instructor now, not at 15:00.

Time budget

Part	Step	Minutes
Part 1 (11:00–12:30)	1. PricingService tests, by hand	20
	2. PricingService implementation	15
	3. Register and login	20
	4. Cities and quotes	25
	Checkpoint 1	10
Part 2 (14:15–15:45)	5. Book a shipment	25
	6. List, show and the policy	20
	7. Cancel and tracking	20
	8. AI code review checklist	15
	Checkpoint 2 and commit	10

Response conventions (read once, use all day)

Every lab this week uses the same JSON shapes. Put this table next to your editor.

Situation	Status	Body
Register	201	`{"token": "...", "user": {...}}`
Login	200	`{"token": "...", "user": {...}}`
Single resource	200 / 201	`{"data": {...}}`
Paginated list	200	`{"data": [...], "links": {...}, "meta": {...}}`
Validation error	422	`{"message": "...", "errors": {"field": ["..."]}}`
No / bad token	401	`{"message": "Unauthenticated."}`
Not allowed	403	`{"message": "This action is unauthorized."}`

Situation	Status	Body
Not found	404	`{"message": "..."}`

Money (`price_lyd`) is always a **string** with two decimals, for example `"322.00"`. Never a JSON float.

> **TIP**
> Always send `-H "Accept: application/json"` with curl. Without it, Laravel treats a failed validation or a missing token as a browser request and answers with a redirect instead of JSON.

Part 1 — Pricing, Auth, Quotes (11:00–12:30)

Step 1 — Write the PricingService tests by hand (20 min)

What and why. Pricing is money. If the agent gets it wrong, a customer is overcharged or Shahna loses money on every trip. The rule is short enough that *you* can write its tests, and you should: the tests are the specification.

> **WARNING**
> **By hand.** Write this test file yourself. Do not ask the agent to write it or to "suggest cases".

The rule (from `docs/requirements.md`):

```
price = 50 LYD base + km × rate_per_km[vehicle_type] + weight_kg × 0.02
rate_per_km: van 0.8, box_truck 1.2, flatbed 1.5, reefer 1.9
Round to 2 decimals. Same city = 0 km.
```

The service has two public methods:

- `price(int $originCityId, int $destinationCityId, VehicleType $vehicleType, int $weightKg): string` — looks up the distance in `city_distances` (0 km for the same city) and returns the price as a string, for example `"322.00"`.
- `createQuote(User $user, int $originCityId, int $destinationCityId, VehicleType $vehicleType, int $weightKg): Quote` — saves a quote with that price, valid for 15 minutes.

The distances come from your Day 1 `CitySeeder`, so the test boots Laravel and seeds the cities. Add this helper to the end of `tests/Pest.php` first:

```php
function city(string $code): App\Models\City
{
    return App\Models\City::where('code', $code)->firstOrFail();
}
```

Work out the expected values on paper first, then create `tests/Unit/PricingServiceTest.php`:

```php
<?php

use App\Enums\VehicleType;
use App\Services\PricingService;
use Database\Seeders\CitySeeder;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

// Pricing reads distances from the database, so these tests boot Laravel.
uses(TestCase::class, RefreshDatabase::class);

beforeEach(function () {
    $this->seed(CitySeeder::class);
    $this->pricing = new PricingService;
});

it('prices Tripoli to Misrata by box truck', function () {
    // 50 + 210 km × 1.2 + 1000 kg × 0.02 = 322
    expect($this->pricing->price(city('TIP')->id, city('MRA')->id, VehicleType::BoxTruck,
        1000))->toBe('322.00');
});

it('uses the rate for each vehicle type', function (VehicleType $type, string $expected) {
    // Tripoli → Zawiya is 45 km, 500 kg adds 10 LYD
    expect($this->pricing->price(city('TIP')->id, city('ZAW')->id, $type, 500))
        ->toBe($expected);
})->with([
    [VehicleType::Van, '96.00'],
    [VehicleType::BoxTruck, '114.00'],
    [VehicleType::Flatbed, '127.50'],
    [VehicleType::Reefer, '145.50'],
]);

it('treats the same city as zero km', function () {
    expect($this->pricing->price(city('BEN')->id, city('BEN')->id, VehicleType::Van, 100))
        ->toBe('52.00');
});

it('prices a long trip without a thousands separator', function () {
    // 50 + 1020 km × 1.9 + 2500 kg × 0.02 = 2038
    expect($this->pricing->price(city('TIP')->id, city('BEN')->id, VehicleType::Reefer, 2500))
        ->toBe('2038.00');
});

it('rounds to two decimals', function () {
    // 50 + 45 × 0.8 + 333 × 0.02 = 92.66
    expect($this->pricing->price(city('TIP')->id, city('ZAW')->id, VehicleType::Van, 333))
        ->toBe('92.66');
});
```

Check your arithmetic for the first case: 50 + 210 × 1.2 + 1000 × 0.02 = 50 + 252 + 20 = **322.00**. Do the others yourself before you trust them. These numbers assume the seeded distances TIP–MRA 210 km, TIP–ZAW 45 km and TIP–BEN 1020 km; if your seeder differs, use your values.

Run it. It must fail, because the class does not exist yet:

```
sail artisan test --filter=PricingService
```

```
  FAILED  Tests\Unit\PricingServiceTest > it prices a trip with ...
  Error: Class "App\Services\PricingService" not found
```

Step 2 — Implement PricingService with the agent (15 min)

What and why. Now the agent writes code against *your* tests. It may not edit them.

AGENT PROMPT

Agent prompt

```
Plan first, then wait for my approval.

Create App\Services\PricingService so that tests/Unit/PricingServiceTest.php
passes. Do NOT modify that test file.
- price(int $originCityId, int $destinationCityId, VehicleType $vehicleType,
  int $weightKg): string
  base 50 + km * rate + weight * 0.02, rates van 0.8, box_truck 1.2,
  flatbed 1.5, reefer 1.9. Return a string with exactly 2 decimals.
- distanceKm(int $originCityId, int $destinationCityId): int
  0 when origin and destination are the same city, otherwise the
  CityDistance row (valueOrFail, so a missing distance throws).
- createQuote(User $user, int $originCityId, int $destinationCityId,
  VehicleType $vehicleType, int $weightKg): Quote
  creates the user's quote with that price and expires_at = now + 15 minutes.
Put the fee and rates in class constants (BASE_FEE_LYD, RATE_PER_KM_LYD,
RATE_PER_KG_LYD, QUOTE_TTL_MINUTES), not in config.
Run `sail artisan test --filter=Pricing` and show the output.
```

What to check.

- Good: the result is formatted with `number_format($total, 2, '.', '')`, which gives `"322.00"`, not `322` or `"322.0"`.
- Good: rates live in one constant array keyed by vehicle type. One place to change prices.
- Red flag: `git diff tests/Unit/PricingServiceTest.php` shows any change. Revert it.
- Red flag: `number_format($total, 2)` without the separator arguments. For prices of 1,000 LYD and above it returns `"2,038.00"`, and your long-trip case fails. Good: the test caught it.

```
sail artisan test --filter=Pricing
```

```
  PASS  Tests\Unit\PricingServiceTest
✓ it prices Tripoli to Misrata by box truck
✓ it uses the rate for each vehicle type with ...
...
✓ it rounds to two decimals

Tests:    8 passed
```

Commit:

```
git add app/Services tests
git commit -m "feat: pricing service"
```

Step 3 — Register and login (20 min)

What and why. Sanctum issues opaque API tokens. The client sends them as `Authorization: Bearer <token>`. Authentication answers *who are you*; tomorrow's policies answer *what may you do*.

AGENT PROMPT

Agent prompt

```
Plan first, then wait for my approval. Tests first, then code.

1. Write Pest feature tests in tests/Feature/AuthTest.php:
   - register with name, email, password (min 8) returns 201
     and JSON {"token": string, "user": {id, name, email, role}} with
     user.role = "customer", and the user's role in the database is customer
     even if the request sends role=admin.
   - register with a duplicate email returns 422 with errors.email.
   - login with correct credentials returns 200 {"token": string, "user": {...}}.
   - login with a wrong password returns 422 with errors.email.
   - GET /api/shipments without a token returns 401.
   Run them and show me they fail.
2. Then implement AuthController with register and login, using FormRequest
   classes, Hash, and $user->createToken('api')->plainTextToken.
   role is not fillable: register sets $user->role = UserRole::Customer itself.
   Controllers live in App\Http\Controllers\Api.
   Routes in routes/api.php: POST /auth/register, POST /auth/login.
   Add an empty auth:sanctum route group for the endpoints that follow.
Small diff. Never return the password hash.
```

What to check.

- Good: `role` is **not** in the `RegisterRequest` rules and not taken from input. Mass assignment of `role` would let anyone make themselves `admin`.
- Good: login failure uses `ValidationException::withMessages(['email' => ...])`, which produces the standard 422 shape.
- Red flag: login says "user not found" for an unknown email but "wrong password" for a known one. That tells an attacker which emails are registered. Both cases must give the same message.

- Red flag: the agent installs Sanctum or publishes its config again. It is already installed on `day1-start`.

Try it by hand:

```
curl -s -X POST http://localhost/api/auth/register \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"name":"Amal","email":"amal@example.ly","password":"secret-pass-1"}'
```

```
{"token":"1|q8V3cX...","user":
        {"id":4,"name":"Amal","email":"amal@example.ly","role":"customer"}}
```

`role` is `customer` because `register` sets it on the server. It is not fillable, so a `role` field in the request body is ignored.

Save the token in a shell variable for the rest of the lab:

```
TOKEN=$(curl -s -X POST http://localhost/api/auth/login \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"email":"amal@example.ly","password":"secret-pass-1"}' | jq -r .token)
echo $TOKEN
```

If you do not have `jq`, copy the token by hand: `TOKEN=2|abc...`.

Step 4 — Cities and quotes (25 min)

What and why. `POST /api/quotes` is the first endpoint that takes real user input. Validation happens at the boundary, in a `FormRequest`, before any of your code runs. A quote is valid for 15 minutes.

AGENT PROMPT

Agent prompt

```
Plan first, then wait for my approval. Tests first, then code, small diffs.

1. GET /api/cities (public, no token): returns {"data": [...]} with id, code,
   name_en, name_ar for all 8 cities, ordered by name_en. Use a CityResource.
2. POST /api/quotes (auth:sanctum), body:
   {origin_city_id, destination_city_id, weight_kg, vehicle_type}
   - StoreQuoteRequest: both city ids required and exist:cities,id;
     weight_kg required integer min:1 max:25000 (the largest truck);
     vehicle_type required and Rule::enum(VehicleType::class).
   - The quote comes from PricingService::createQuote(), never from the
     request; user_id is the authenticated user; expires_at = now + 15 min.
   - Return 201 with QuoteResource: id, origin and destination as
     CityResource {id, code, name_en, name_ar}, vehicle_type, weight_kg,
     price_lyd as a string with 2 decimals, expires_at ISO 8601.
Feature tests in tests/Feature/QuoteTest.php: happy path with price check,
unknown city → 422, bad vehicle_type → 422, weight 0 → 422,
price_lyd sent in the body is ignored, unauthenticated → 401.
Use Carbon::setTestNow / $this->freezeTime() for expires_at.
```

What to check.

- Good: `Rule::enum(VehicleType::class)` instead of a hand-typed `in:van,box_truck,...` list that will drift from the enum.
- Good: the controller uses `$request->validated()`, never `$request->all()`.
- Good: `QuoteResource` eager-loads or receives `origin` and `destination`; it does not run a query per field.
- Red flag: `price_lyd` is in the FormRequest rules. A customer would choose their own price.
- Red flag: the test asserts `expires_at` with `now()` without freezing time. It will fail one run in a hundred.

```
curl -s http://localhost/api/cities \
  -H "Accept: application/json" | jq '.data[0]'
```

```
{"id":2,"code":"BEN","name_en":"Benghazi","name_ar":"بنغازي"}
```

Get a box-truck quote from Tripoli to Misrata. Look up their ids in the cities list first (with the reference seeder TIP is 1 and MRA is 3):

```
curl -s -X POST http://localhost/api/quotes \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"origin_city_id":1,"destination_city_id":3,"weight_kg":1000,"vehicle_type":"box_truck"}'
```

```
{
  "data": {
    "id": 1,
    "origin": {"id": 1, "code": "TIP", "name_en": "Tripoli", "name_ar": "طرابلس"},
    "destination": {"id": 3, "code": "MRA", "name_en": "Misrata", "name_ar": "مصراتة"},
    "vehicle_type": "box_truck",
    "weight_kg": 1000,
    "price_lyd": "322.00",
    "expires_at": "2026-10-21T11:52:10.000000Z"
  }
}
```


(curl shows the Arabic names as `\u` escapes; `jq` prints them as Arabic.) Your `price_lyd` is `"322.00"` only if your seeded TIP to MRA distance is 210 km. Otherwise compute what it should be and check.

Now a bad request:

```
curl -s -X POST http://localhost/api/quotes \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"origin_city_id":1,"destination_city_id":999,"weight_kg":0,"vehicle_type":"rocket"}'
```


```
{
  "message": "The selected destination city id is invalid. (and 2 more errors)",
  "errors": {
    "destination_city_id": ["The selected destination city id is invalid."],
    "vehicle_type": ["The selected vehicle type is invalid."],
    "weight_kg": ["The weight kg field must be at least 1."]
  }
}
```

CHECKPOINT

Checkpoint 1 (12:30)

```
sail artisan test
```

```
  Tests:    ~38 passed
```

Your count will differ (the full `day2-end` reference ends the day at 56); **zero failures** is what matters.

Register returns 201 with a token; login returns 200 with a token.

`GET /api/cities` works without a token; `GET /api/shipments` without a token returns 401 `{"message":"Unauthenticated."}`.

`POST /api/quotes` returns 201 with `price_lyd` as a string.

```
git add -A
git commit -m "feat: auth, cities and quotes endpoints"
```

Part 2 — Shipments, Policies, Review (14:15–15:45)

Step 5 — Book a shipment (25 min)

What and why. Booking turns a quote into a shipment. This is where business rules live: a quote can be used once, only by its owner, and only before it expires. The application checks these rules and returns friendly 422s, with the quote row locked (`lockForUpdate()`) inside the transaction; the database's **unique index on** `shipments.quote_id` (from Day 1) is the final backstop. Tomorrow you will add idempotency keys so a retried request is safe too.

AGENT PROMPT

Agent prompt

```
Plan first, then wait for my approval. Tests first: write them, run them,
show me they fail, then implement. Small diffs.

POST /api/shipments (auth:sanctum), body {quote_id, pickup_at}.
StoreShipmentRequest:
- quote_id required, integer
- pickup_at required, date, after:now
Put the logic in App\Services\BookingService::book(User $user, int $quoteId,
CarbonInterface $pickupAt): Shipment. Inside DB::transaction, load the quote
with Quote::whereBelongsTo($user)->lockForUpdate()->find($quoteId).
Business rules (throw ValidationException with errors.quote_id when violated):
- the quote belongs to the authenticated user (otherwise 422, do not
  reveal that the quote exists for someone else)
- the quote is not expired (expires_at > now)
- the quote has not already been used by a shipment
On success, inside the same transaction:
- create the Shipment copying origin, destination, weight_kg,
  vehicle_type and price_lyd from the quote (never from the request),
  status pending, reference generated by the model
- create a ShipmentEvent with status pending, note "Booking received."
- add Shipment::transitionTo(ShipmentStatus $next, ?string $note) that checks
  canTransitionTo (ValidationException on errors.status if not allowed),
  updates the status and writes a ShipmentEvent; use it to move the new
  shipment to confirmed with note "Booking confirmed."
Return 201 with ShipmentResource: reference, status, origin, destination,
vehicle_type, vehicle (null until assigned), weight_kg, price_lyd (string),
pickup_at, created_at.
Tests in tests/Feature/ShipmentTest.php: happy path, expired quote,
already-used quote, another user's quote, pickup_at in the past,
and a test proving the DB unique index rejects a second shipment with the
same quote_id (insert directly, expect QueryException).
```

What to check.

- Good: price is copied from the quote. Read the controller: if `price_lyd` comes from `$request` anywhere, that is a critical bug.
- Good: the “used” check is `Shipment::where('quote_id', ...)->exists()` or a `hasOne` relation on `Quote`, plus the DB unique index.
- Good: the expired-quote test uses `$this->travel(16)->minutes()` instead of sleeping.
- Red flag: the other-user’s-quote case returns 403 or 404 with “quote belongs to another user”. That leaks information. Treat it exactly like a non-existent quote.
- Red flag: the shipment and its first event are created outside a transaction. If the event insert fails, you have a shipment with no history.

Book the quote you created in Step 4:

```
curl -s -X POST http://localhost/api/shipments \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"quote_id":1,"pickup_at":"2026-10-25T08:00:00Z"}'
```


```
{
  "data": {
    "reference": "SHN-7KQ2ZD",
    "status": "confirmed",
    "origin": {"id": 1, "code": "TIP", "name_en": "Tripoli", "name_ar": "طرابلس"},
    "destination": {"id": 3, "code": "MRA", "name_en": "Misrata", "name_ar": "مصراتة"},
    "vehicle_type": "box_truck",
    "vehicle": null,
    "weight_kg": 1000,
    "price_lyd": "322.00",
    "pickup_at": "2026-10-25T08:00:00.000000Z",
    "created_at": "2026-10-21T11:40:02.000000Z"
  }
}
```


If you get `"This quote has expired. Please request a new quote."`, it has been more than 15 minutes since Step 4. Create a new quote. That is the rule working.

Send the same request again. You must get a 422, not a second shipment:


```
{"message":"This quote has already been booked.","errors":{"quote_id":["This quote has already
        been booked."]}}
```


Save the reference: `REF=SHN-7KQ2ZD`.

Step 6 — List, show and the policy (20 min)

What and why. A customer sees only their own shipments. Staff (a dispatcher or an admin) can see any of them. This rule belongs in **one** place, a policy, not scattered across controllers as `if` statements. Laravel 12's base `Controller` does not include the `AuthorizesRequests` trait, so you call `Gate::authorize('view', $shipment)`.

Public URLs use the shipment **reference**, never the numeric id. Route model binding does that for you with `{shipment:reference}`.

First, write one policy test **by hand**.

> **WARNING**
>
> **By hand.** Write the test below yourself before you prompt the agent. It is the authorization rule in its smallest form.

Create `tests/Feature/ShipmentPolicyTest.php`:

```php
<?php

use App\Enums\UserRole;
use App\Models\Shipment;
use App\Models\User;

it('lets the owner view their shipment', function () {
    $shipment = Shipment::factory()->create();

    $this->actingAs($shipment->user, 'sanctum')
        ->getJson("/api/shipments/{$shipment->reference}")
        ->assertOk()
        ->assertJsonPath('data.reference', $shipment->reference);
});

it('forbids another customer', function () {
    $shipment = Shipment::factory()->create();
    $stranger = User::factory()->create(['role' => UserRole::Customer]);

    $this->actingAs($stranger, 'sanctum')
        ->getJson("/api/shipments/{$shipment->reference}")
        ->assertForbidden();
});
```

(Use `RefreshDatabase`: check that `tests/Pest.php` already applies it to the `Feature` folder; if not, add `->use(Illuminate\Foundation\Testing\RefreshDatabase::class)`.)

Run it and watch it fail with 404 (the route does not exist yet). Now hand over to the agent:

AGENT PROMPT

Agent prompt

```
Plan first, then wait for my approval. Do not modify
tests/Feature/ShipmentPolicyTest.php; make it pass and add to it.

1. App\Policies\ShipmentPolicy with view(User, Shipment): true for the owner
   or a staff user. Add User::isStaff(): role is dispatcher or admin.
   Use the UserRole enum, not strings.
2. GET /api/shipments/{shipment:reference} → Gate::authorize('view', $shipment),
   returns ShipmentResource.
3. GET /api/shipments → the authenticated user's own shipments only,
   newest first, paginated 15 per page, ShipmentResource::collection.
Add tests: dispatcher can view any shipment; unknown reference → 404;
list only contains my shipments (create shipments for 2 users and assert
count and references); list is paginated (meta.per_page = 15).
```

What to check.

- Good: the list query starts from `$request->user()->shipments()`, so other users' rows can never appear.
- Good: the policy compares ids: `$user->id === $shipment->user_id`.

- Red flag: the list endpoint uses `Shipment::paginate()` and then filters in PHP, or filters by a `user_id` query parameter. Both leak data.
- Red flag: the policy compares raw strings (`$user->role === 'dispatcher'`) instead of going through `isStaff()` and the `UserRole` enum.
- Note: the list will run one query per shipment for `origin`, `destination` and `vehicle`. That is the N+1 problem. **Leave it.** You will measure and fix it tomorrow.

```
curl -s http://localhost/api/shipments \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" | jq '{count: (.data|length),
        meta: .meta.per_page}'
```

```
{"count": 1, "meta": 15}
```

(Amal has one shipment. The `day2-end` reference also adds a `DemoShipmentSeeder` that gives `customer@shahna.test` 30 confirmed shipments, including the fixed reference `SHN-DEMO01`; later labs use it.)

Register a second user, get their token as `TOKEN2`, and try to read Amal's shipment:

```
curl -s http://localhost/api/shipments/$REF \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN2"
```

```
{"message":"This action is unauthorized."}
```

Step 7 — Cancel and tracking (20 min)

What and why. Cancelling reuses the rule you wrote and tested yesterday: `ShipmentStatus::canTransitionTo()`. Never re-implement it in the controller. Every status change also writes a `ShipmentEvent`, which is what the tracking endpoint shows.

AGENT PROMPT

Agent prompt

```
Plan first, then wait for my approval. Tests first, small diffs.

1. Add cancel(User, Shipment) to ShipmentPolicy: the owner or staff.
2. POST /api/shipments/{shipment:reference}/cancel:
   Gate::authorize('cancel', $shipment), then BookingService::cancel():
   in a transaction, re-read the shipment with
   Shipment::lockForUpdate()->findOrFail($shipment->id) and call
   transitionTo(ShipmentStatus::Cancelled, 'Cancelled by customer.').
   transitionTo already returns 422 with errors.status when the move is
   not allowed. Return 200 with ShipmentResource.
3. GET /api/shipments/{shipment:reference}/tracking: Gate::authorize('view'),
   returns ShipmentEventResource::collection of events ordered by
   occurred_at ascending: status, note, occurred_at.
Tests: cancel confirmed → 200 and an event exists; cancel in_transit → 422
and status unchanged; cancel already cancelled → 422; another customer
→ 403; tracking returns events in order; tracking for another customer's
shipment → 403; cancelling a stale copy re-checks the latest status.
Use ShipmentFactory's status() state.
```

What to check.

- Good: the status rule is checked through `transitionTo()` / `canTransitionTo()`; there is no list of statuses in the controller.
- Good: the in-transit test asserts the status is *still* `in_transit` in the database after the 422 (`$shipment->refresh()`).
- Red flag: the 422 is thrown as `abort(422, ...)`. That gives `{"message": ...}` without `errors`, which breaks the convention table. Use `ValidationException::withMessages`.
- Red flag: the policy and the 422 check happen in the wrong order. A stranger must get 403, not 422 (otherwise they learn the shipment's status).

```bash
curl -s -X POST http://localhost/api/shipments/$REF/cancel \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" | jq '.data.status'
```

```
"cancelled"
```

```bash
curl -s http://localhost/api/shipments/$REF/tracking \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN"
```

```
{
  "data": [
    {"status": "pending", "note": "Booking received.", "occurred_at": "2026-10-
        21T11:40:02.000000Z"},
    {"status": "confirmed", "note": "Booking confirmed.", "occurred_at": "2026-10-
        21T11:40:02.000000Z"},
    {"status": "cancelled", "note": "Cancelled by customer.", "occurred_at": "2026-10-
        21T14:58:40.000000Z"}
  ]
}
```


Cancel it again:


```
{"message":"A cancelled shipment cannot move to cancelled.","errors":{"status":["A cancelled
        shipment cannot move to cancelled."]}}
```


Step 8 — AI code review checklist (15 min)

What and why. You have accepted a lot of agent code today. Now review it as if a colleague opened a pull request. First **by hand**, then with the agent, then compare.

> **WARNING**
>
> **By hand first.** Spend 8 minutes on `git diff day1-end --stat` and then read the controllers, requests and policy. Fill in the checklist below on paper before you prompt the agent.

#	Check	Where to look
1	Every input is validated in a FormRequest; controllers use `validated()`	`app/Http/Requests`, controllers
2	No mass assignment of `role`, `user_id`, `price_lyd`, `status` from input	requests, `$fillable`
3	Every route except register, login and cities is behind `auth:sanctum`	`sail artisan route:list --path=api`
4	Every shipment read/write is authorized by the policy	controllers
5	Money is `decimal`, serialized as a string	migrations, resources
6	Multi-row writes are in `DB::transaction`	`app/Services/BookingService.php`

#	Check	Where to look
7	Errors follow the convention table (422 has `errors`)	tests
8	Tests cover the unhappy paths, not just the happy one	`tests/Feature`
9	No test was weakened to make code pass	`git log -p tests/`
10	Nothing was added that the spec did not ask for	whole diff

```
sail artisan route:list --path=api
```

```
  POST       api/auth/login ......................... Api\AuthController@login
  POST       api/auth/register ...................... Api\AuthController@register
  GET|HEAD   api/cities ............................. Api\CityController@index
  POST       api/quotes ............................. Api\QuoteController@store
  GET|HEAD   api/shipments ...................... Api\ShipmentController@index
  POST       api/shipments ...................... Api\ShipmentController@store
  GET|HEAD   api/shipments/{shipment} ............ Api\ShipmentController@show
  POST       api/shipments/{shipment}/cancel ... Api\ShipmentController@cancel
  GET|HEAD   api/shipments/{shipment}/tracking Api\ShipmentController@tracking
```

Then run the same review through the agent:

AGENT PROMPT

Agent prompt

```
Do not change any files. Review `git diff day1-end` as a senior reviewer
using this checklist: validated input only; no mass assignment of role,
user_id, price_lyd or status; auth:sanctum on every route except register, login and
cities;
policy on every shipment read and write; money as decimal and string;
transactions around multi-row writes; error shape matches Laravel defaults;
unhappy-path tests exist; no test was weakened; no unrequested features.
For each item answer PASS or FAIL with file:line evidence.
```

Compare the agent's answers with yours. Wherever you disagree, open the file and decide who is right. Fix real FAILs in small, separate commits.

CHECKPOINT

Checkpoint 2 (15:45)

```
sail artisan migrate:fresh --seed
sail artisan test
```

```
   PASS  Tests\Unit\PricingServiceTest
   PASS  Tests\Unit\ShipmentStatusTest
   PASS  Tests\Feature\AuthTest
   PASS  Tests\Feature\CityTest
   PASS  Tests\Feature\QuoteTest
   PASS  Tests\Feature\ShipmentPolicyTest
   PASS  Tests\Feature\ShipmentTest
   ...
  Tests:    56 passed
```

The `day2-end` reference has **56** tests (23 from Day 1, 33 new). You need **at least 20 new tests today** and zero failures; your total will differ.

End-to-end smoke test (fresh database, so register again):

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/api/shipments -H "Accept:
        application/json"
# 401
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/api/shipments -H "Accept:
        application/json" -H "Authorization: Bearer $TOKEN"
# 200
```

```
git add -A
git commit -m "feat: shipments, policy, cancel and tracking"
```

If you're stuck

The reference solution is the `day2-end` tag.

```
# Which files differ from the reference?
git diff --stat day2-end

# Read the reference version of a file without switching branches
git show day2-end:app/Policies/ShipmentPolicy.php

# Diff one file against the reference
git diff day2-end -- app/Http/Controllers/Api/ShipmentController.php

# Run the reference tests against your code (copy one test file over)
git show day2-end:tests/Feature/ShipmentTest.php > tests/Feature/ShipmentRefTest.php
sail artisan test --filter=ShipmentRef
```

That last trick is powerful: the reference tests are a free specification. If they fail on your code, read why.

To restart from the reference (your work stays on `my-day2`):

```
git stash
git switch -c my-day2-reset day2-end
sail artisan migrate:fresh --seed
sail artisan test
```

Stretch goals

1. **Check constraints.** Add PostgreSQL `CHECK (price_lyd ≥ 0)` and `CHECK (weight_kg > 0)` constraints on `quotes` and `shipments`, and a test that proves the database rejects bad rows even when the application is bypassed.
2. **Integer money.** Change `PricingService` to compute in integer dirhams (or use `bcmath`) instead of floats, keeping every test in Step 1 green. Explain in one sentence why floats are risky for money.
3. **Logout.** `POST /api/auth/logout` deletes only the current token (`$request→user()→currentAccessToken()→delete()`). Test that the old token then returns 401.
4. **Filtering.** `GET /api/shipments?status=pending` using `Rule::enum` validation on the query string. Test that an invalid status gives 422.
5. **Staff list.** Dispatchers and admins see all shipments in `GET /api/shipments`. Decide whether this should be the same endpoint or a new `/dispatch/shipments` route, and justify it.

Reflection

1. Which test did the agent try to change, skip, or weaken today, if any? If none, which test did you read most carefully, and why?
2. Find one place where the agent's code was correct but did not follow the response conventions table (status code, error shape, money as string). How did you find it: a test, curl, or reading?

3. In Step 8, where did your by-hand review and the agent's review disagree? Who was right?